KAIROS: Proactive Causal Root-Cause Analysis of Business Metrics with Verified Natural-Language Alerts

Palla Chetana Reddy
Dept. of Computer Science and Engineering
School of Technology, Woxsen University
Hyderabad, India

Varshini Yadav Nasaram
Dept. of Computer Science and Engineering
School of Technology, Woxsen University
Hyderabad, India

Obulagari Nandini
Dept. of Computer Science and Engineering
School of Technology, Woxsen University
Hyderabad, India

***Abstract*—Dashboards in business intelligence (BI) tools wait for a person to ask a question, and the explanations they offer for a metric change are correlational. This paper presents KAIROS, a five-agent system that monitors business metrics from heterogeneous sources on a schedule, detects anomalies, identifies their causal origin and delivers a plain-English alert whose numbers have been checked against the data. Anomalies are scored with a robust seasonal z-score. A Causal Business Metric Graph (CBMG) is learned with DYNOTEARS, a continuous-optimization method for lagged structure learning, and root-cause chains are ranked by path strength, bootstrap stability and the timing of anomalies. Reports are drafted by a language model and released only if an LLM-Modulo critic finds every number and metric name supported by computed facts. On a synthetic benchmark of 15 metrics with a known causal graph, DYNOTEARS reached an F1 score of 0.97 ± 0.02 over ten seeds, compared with 0.90 for lagged Granger regression, 0.87 for VARLiNGAM and 0.50 for correlation, and the pipeline named the true origin of both injected incidents in every run. The critic rejected 98.5% of reports containing a corrupted number while accepting every faithful one. The system is open source and runs on a laptop.**

***Index Terms*—business intelligence, anomaly detection, causal discovery, root cause analysis, DYNOTEARS, large language models, verification.**

I. Introduction

Business intelligence platforms have made company data easy to look at, but not easy to watch. A manager opens a dashboard, reads the charts and decides whether anything is wrong. When nobody opens it, a fault such as a paused advertising campaign or a warehouse backlog can run for a whole weekend before it is noticed. The explanation features that vendors have added rank fields that move together with the selected metric [1]. That is useful as a hint, but it cannot say which metric moved first, and in a chain of connected KPIs nearly everything moves together.

Three lines of work make a different arrangement possible. Robust statistics and decomposition models detect unusual points in seasonal series without labels [2], [3]. Continuous optimization has turned the search for a directed acyclic graph into a smooth problem [4], and DYNOTEARS extends it to lagged time-series structure [5]. Large language models (LLMs) write readable summaries [6], although they also state facts that are not in their input [7]. Agent frameworks built on the ReAct pattern [8] and graph-based orchestration [9] make it straightforward to connect such steps into one loop.

Putting these parts together raises two practical questions. Can causal discovery on a few weeks of observational KPI data point to where an incident started, rather than to whatever correlates with it? And can a machine-written explanation be trusted enough to send to a manager with nobody checking it first? This paper addresses both with KAIROS and a benchmark whose causal structure is known. Its contributions are:

- A proactive monitoring pipeline in which anomalies are persisted and gated, so that each incident triggers one alert rather than one per cycle.
- A ranking of root-cause chains on a DYNOTEARS-learned Causal Business Metric Graph (CBMG) that combines path strength, bootstrap edge stability and anomaly timing to name an origin.
- A precision-aware LLM-Modulo critic [10] for numeric claims in generated reports, together with a stress test of how often it catches corrupted numbers.
- A reproducible comparison with three discovery baselines; the code and scripts that regenerate every number are released with the system.

II. Related Work

A. Monitoring and Anomaly Detection

Anomaly detection has a long literature [11]. Prophet fits trend and seasonal components and gives prediction intervals [3]. Deep detectors such as the Anomaly Transformer learn association discrepancies between time points [12], but they need training on each dataset and long histories. For business metrics that arrive hourly and change slowly, median-based scores remain attractive because the anomalies being sought do not inflate the scale estimate [2]. Commercial BI tools mark outliers on individual charts, yet a person still has to open the chart.

B. Causal Discovery and Root-Cause Analysis

Granger causality tests whether the past of one series improves the prediction of another [13]. PCMCI combines lagged conditional-independence tests with a selection step [14], and VARLiNGAM identifies a structural vector autoregression from non-Gaussian noise [15]. NOTEARS replaced combinatorial search with a smooth acyclicity constraint [4], and DYNOTEARS learns contemporaneous and

lagged weights together [5]. Root-cause localization in operations (AIOps) uses similar ideas: CloudRanger [16], MicroCause [17] and RCD [18] rank faulty services from telemetry. Those systems target engineers and system metrics with second-level resolution. Business KPIs are fewer, slower and strongly seasonal, and the people who act on them need an explanation in words. Effect estimation given a graph is a separate step, supported by libraries such as DoWhy [19]; interventional claims ultimately rest on assumptions about the graph [20].

C. LLM Narration and Verification

Hallucination is well documented for text generation [7]. Kambhampati et al. argue that LLM output should be treated as a candidate that sound external critics accept or reject [10]. Data-to-text reporting is a favourable case for this design, because most claims in a metric alert are numbers that can be checked exactly against computed facts.

III. System Overview

Fig. 1 shows the architecture. Five agents run as nodes of a LangGraph state machine [9], started every 15 minutes by a background scheduler. Agent 1 pulls new data from SQL databases, SFTP/FTP folders, local or cloud files and REST APIs, or accepts pushed records, and converts it to rows of (metric, value, timestamp). Aggregation is pushed down to the source where possible, so only KPI rows travel. Rows are upserted into DuckDB [21] under the key (source, metric, timestamp), which makes repeated syncs harmless. Agent 2 scores every series, Agent 3 builds the CBMG and ranks causes, Agent 4 writes and verifies the report, and Agent 5 sends it through Telegram [22], e-mail and the dashboard.

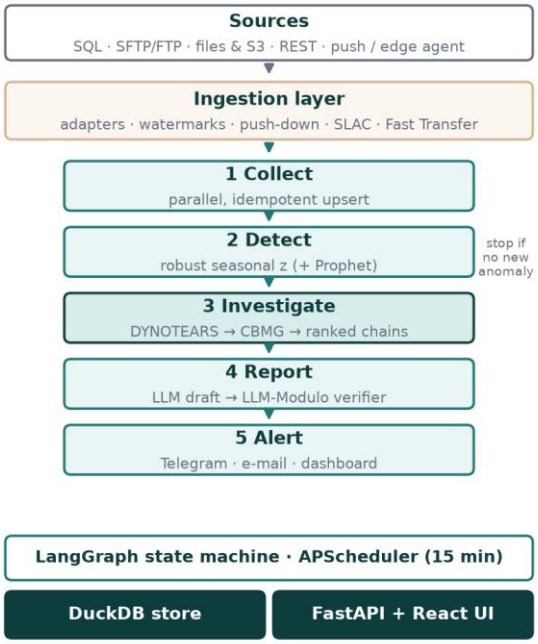


Fig. 1. KAIROS architecture. The graph ends after detection when no alert-worthy anomaly is new.

An anomaly is alert-worthy if it is seen for the first time, lies within 24 hours of the newest point of its series, has at least medium severity, and its metric has not been alerted in the last 60 minutes. When no anomaly qualifies, the graph ends after detection, so a quiet cycle costs well under a second.

IV. Methods

A. Robust Seasonal Detection

For a series x_t with n points, the expected value m_t is a centred rolling median with window $w = \min(25, \max(5, n/4))$ plus an hour-of-day profile of the residual medians. The profile is kept only if it lowers the residual median absolute deviation (MAD) by at least 10%. The score is

$$z_t = (x_t - m_t) / (1.4826 \cdot MAD(x - m)) \quad (1)$$

and a point is flagged when $|z_t| \geq 3.5$. Severity is medium from 4.5, high from 6 and critical from 9 robust standard deviations. Prophet can run as a second detector; its residual is divided by the scale implied by its 99% interval so that both detectors report on the same scale.

B. Learning the Causal Business Metric Graph

Metrics are aligned on a common grid at their median sampling step, gaps of up to three steps are interpolated, and values are clipped at the median ± 3.5 robust standard deviations so that the incident being explained does not dominate the fitted structure. After standardization, DYNOTEARS fits the structural vector autoregression

$$X_t = X_t W + X_{t-1} A + \varepsilon \quad (2)$$

where W holds same-step effects and A one-step lagged effects. It solves

$$\min (1/2n)\|X_t - X_t W - X_{t-1} A\|^2 + \lambda_W \|W\|_1 + \lambda_A \|A\|_1 \quad (3)$$

subject to the acyclicity constraint of NOTEARS [4]

$$h(W) = tr(e^{W \circ W}) - d = 0 \quad (4)$$

The problem is solved with an augmented Lagrangian and L-BFGS-B on the positive and negative parts of the weights, with $\lambda_W = \lambda_A = 0.05$. Edges with $|w| \geq 0.15$ form the CBMG. The implementation is about 120 lines of NumPy and SciPy. To estimate stability, the model is refitted on two overlapping windows covering 75% of the series; the confidence c_e of an edge is the share of the three fits that keep it.

C. Ranking Root-Cause Chains

For each anomalous metric y and each ancestor x in the CBMG, the strongest path π from x to y (at most four hops) is found. Its strength $S(\pi)$ is the product of edge weights and its lag $L(\pi)$ the sum of edge lags. The evidence flag $e(\pi)$ is 1 when x itself had an anomaly within $L(\pi) + 2$ steps before an anomaly of y. The origin flag o(x) is 1 when $e(\pi) = 1$ and no ancestor of x was anomalous in the six steps before x. Chains are scored by

$$r(\pi) = S(\pi)^{1/|\pi|} \cdot \bar{c}(\pi) \cdot (1+0.5e(\pi)) \cdot (1+o(x)) \cdot (1+0.05|\pi|) \quad (5)$$

where $\bar{c}$ is the mean edge confidence. Taking the per-hop geometric mean of the strength matters: a plain product penalizes long chains, and the long chains are the ones that reach back to the true origin. The three best chains per affected metric are kept, and for direct edges DoWhy estimates a linear back-door effect with the other parents of y as controls [19].

D. Verified Reporting

The reporter builds a fact sheet: the most severe anomaly of each metric (observed and expected value, percentage change, severity, time), the origins, and the five best chains with strength, confidence and lag. Only these facts go into the prompt. The draft comes from a hosted open model, with a local Ollama model as fallback. The critic then extracts every number. Integers up to 12 written without a percent sign are skipped, because they are list numbering or counts of actions. A token written with d decimals and value v is accepted only if some allowed number a derived from the fact sheet satisfies

$$|v - a| \le 0.5 \cdot 10^{-d} + 10^{-3} \cdot |a| \qquad (6)$$

Every snake_case identifier must also be a known metric. A rejected draft is returned once with the list of problems; a second failure replaces it with a deterministic template. Equation (6) ties the tolerance to the precision the writer chose. An earlier, looser rule that compared rounded integers accepted 7.03 as a match for 6.71, a mistake that the stress test in Section VI exposed.

V. Experimental Setup

The benchmark simulates four company systems (ERP, marketing exports, support desk and logistics) producing 15 hourly metrics over 14 days (336 points). Values follow a lag-one structural autoregression on a standardized scale with 12 true edges arranged in two chains, marketing (advertising spend → click-through → conversion → orders → revenue, with cart abandonment, order value and refunds attached) and operations (inventory turnover → delivery delay → open tickets → response time → NPS → churn). Each metric has an autoregressive coefficient of 0.25, Gaussian noise with standard deviation 0.55, and four metrics carry a daily cycle. Two incidents are injected in the last 24 hours, a three-hour drop of nine standard deviations in advertising spend and a matching rise in delivery delay, which propagate down their chains. Six isolated anomalies between 4 and 12 standard deviations are added elsewhere.

Each experiment runs over ten random seeds. For causal discovery we report precision, recall, F1 and structural Hamming distance (SHD, a reversed edge counted once) of the union of contemporaneous and lagged edges. Baselines use the same 0.15 threshold: lagged least-squares (Granger) coefficients, VARLiNGAM from causal-learn, and Pearson correlation with $|r| \ge 0.3$, counted in both directions because correlation has none. Detection is scored by event recall (an affected metric flagged inside the incident window) and point precision. The critic is tested on five seeds with 200 corrupted numbers, 50 invented metric names and 200 faithful one-decimal roundings. All runs used a laptop with Python 3.13.

VI. Results

A. Causal Discovery

Table I and Fig. 2 summarize discovery quality. DYNOTEARS did not add a single spurious edge in ten runs. Its few misses were the weakest true link, average order value to revenue. Granger regression and VARLiNGAM found nearly every true edge but added three to four false ones on average, typically between metrics that share a driver. Correlation was worst: without direction, every true edge also produces a false reverse edge.

TABLE I
Causal discovery over ten seeds (mean ± s.d.)

Method	Precision	Recall	F1	SHD
DYNOTEARS	1.00 ± 0.00	0.95 ± 0.04	0.97 ± 0.02	0.6 ± 0.5
Granger (OLS)	0.83 ± 0.13	0.99 ± 0.03	0.90 ± 0.08	2.8 ± 2.5
VARLiNGAM	0.78 ± 0.11	0.99 ± 0.03	0.87 ± 0.08	3.7 ± 2.5
Correlation	0.44 ± 0.01	0.58 ± 0.07	0.50 ± 0.03	7.0 ± 0.9

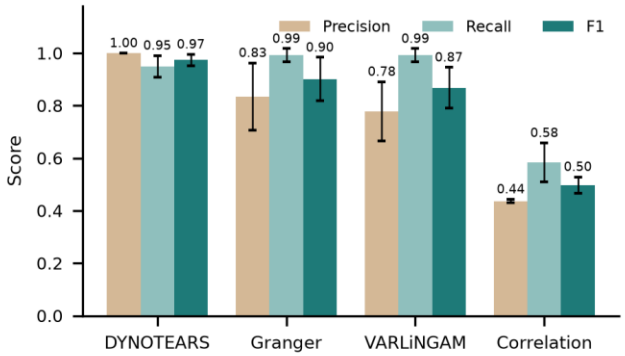


Fig. 2. Precision, recall and F1 of each discovery method (error bars: one standard deviation over ten seeds).

B. Origin Localization and Detection

Run end to end, from detection to ranking, KAIROS reported exactly the two injected origins, advertising spend and delivery delay, in all ten runs (Table II). The investigation, including two bootstrap refits and DoWhy, took 1.58 ± 0.57 s. The detector found every injected event. Most of the flags counted against precision sit just outside the event windows, where the cascade is still settling. Adding Prophet did not change recall, lowered precision slightly and made detection about fourteen times slower, so it is disabled by default.

TABLE II
Detection and origin localization

Measure	Result
Event recall, robust z (10 seeds)	100.0 ± 0.0%
Point precision, robust z	77.2 ± 6.2%
Point precision, robust z + Prophet (5 seeds)	75.2 ± 4.1%
Detection time for 15 series	0.23 s (Prophet: 3.34 s)
Both incident origins named exactly	10 of 10 runs
Investigation time	1.58 ± 0.57 s

C. Report Verification

Table III shows the critic's stress test. It caught 98.5% of corrupted numbers and every invented metric name, while accepting all faithful reports and all faithful one-decimal roundings. The three corruptions it missed landed, by chance, within rounding distance of another true fact. With the earlier rounding rule, only 61% of corrupted numbers were caught. In a live run against four simulated source systems, the hosted

model's report passed on the first attempt, and the full five-agent cycle took 12.4 s.

TABLE III
LLM-MODULO CRITIC STRESS TEST

Test	Cases	Result
Faithful reports accepted	5	100%
Corrupted numbers rejected	200	98.5%
Invented metric names rejected	50	100%
Faithful roundings accepted	200	100%

D. Case Study: Following an Incident

Fig. 3 shows the marketing incident for one seed as KAIROS sees it. Advertising spend collapses at the dashed line; click-through falls one hour later, conversion two hours later, orders three and revenue four. A correlation view would report all five metrics as related to the revenue drop with similar strength. The CBMG instead recovers the chain with one-hour lags, and the ranking in (5) places advertising spend first because it is anomalous and has no anomalous ancestor. The resulting alert names advertising spend as the origin, quotes the observed and expected values, and recommends checking the campaign, which is the action an analyst would reach after tracing the chain by hand.

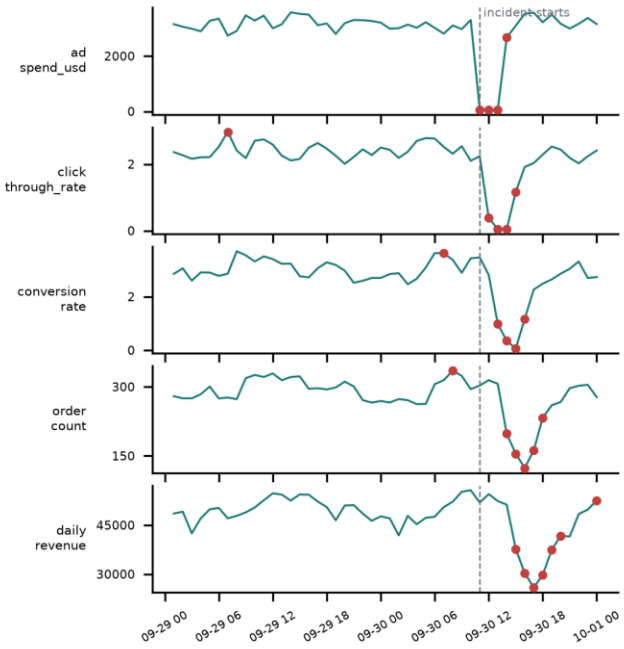


Fig. 3. Marketing incident (seed 3), last 48 h. Red points are flagged anomalies; the dashed line marks the start of the injected drop in advertising spend. The effect reaches revenue four hours later.

E. Sensitivity to Thresholds

The thresholds above were fixed before the experiments. Fig. 4 shows how much they matter. For discovery, the L1 penalty and the edge threshold act as substitutes: with a weak penalty ($\lambda = 0.01$) a low threshold keeps many small spurious weights and F1 drops to 0.57, while a stronger penalty tolerates a lower threshold. The chosen setting ($\lambda = 0.05$, $\tau = 0.15$) gives the highest F1 of the grid, 0.97, and F1 stays above 0.89 for every threshold at that penalty. For detection, raising $|z|$ from 3.5 to 4.0 lifts point precision from 0.77 to 0.93 at the cost of one missed weak event in ten runs (event recall 0.994). We keep 3.5 because a missed incident costs more than a review of a borderline flag, but deployments that prefer fewer alerts can raise it.

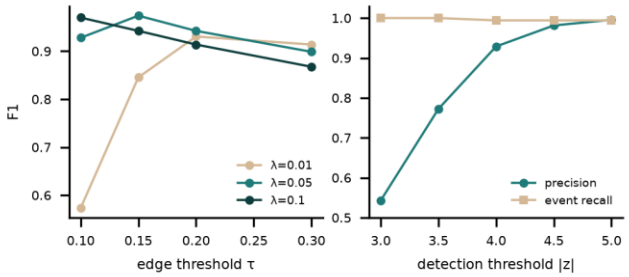


Fig. 4. Left: DYNOTEARS F1 against edge threshold τ for three L1 penalties. Right: detection precision and event recall against the $|z|$ threshold. Means over ten seeds.

VII. DISCUSSION AND LIMITATIONS

The results support the main claim: on data whose structure is known, lagged causal discovery points to the origin of an incident where correlation cannot, and it does so quickly enough to run every cycle. Several limits qualify this. The benchmark follows a linear lag-one model, the same model class DYNOTEARS assumes, so it is a favourable test, and real KPIs will show nonlinear effects and changing regimes. DYNOTEARS also assumes that no unmonitored variable drives two monitored ones. A pricing change made outside the monitored sources could appear as an edge between its effects. The origin KAIROS names is therefore the earliest anomalous monitored metric consistent with the learned graph. That is a starting point for an analyst, not proof of cause. The thresholds were fixed in advance; Section VI-E shows they are reasonable on the benchmark, but they will need recalibration on real data. The critic covers numbers and names only. In one live report the model described every anomaly as critical, which was not true and was not caught; qualitative claims need their own checks. Finally, each graph is capped at 20 variables, which suits a business unit but not a whole enterprise.

VIII. CONCLUSION

KAIROS shows that business monitoring can work as a push system: it finds anomalies, traces them through a learned causal graph to an origin and reports them in language whose numbers have been checked. On a benchmark with known structure, DYNOTEARS recovered the graph with an F1 of 0.97, the pipeline named both incident origins in every run, and the critic stopped nearly all numeric misstatements. Future work will evaluate the system on public retail datasets and partner data, add nonlinear and confounder-aware discovery, extend verification to qualitative claims, and study how managers use the alerts.

ACKNOWLEDGMENT

The authors thank Dr. K RajKumar for guidance throughout the project and the Capstone review panel of the School of Technology, Woxsen University, for comments that shaped the system.

REFERENCES

[1] "Power BI documentation: Key influencers visual," Microsoft. [Online]. Available: https://learn.microsoft.com/power-bi/visuals/power-bi-visualization-influencers. [Accessed: Oct. 6, 2026].

[2] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.

[3] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[4] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[5] R. Pamfil, N. Sriwattanaworachai, S. Desai, P. Pilgerstorfer, K. Georgatzis, P. Beaumont, and B. Aragam, "DYNOTEARS: Structure learning from time-series data," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 108, 2020, pp. 1595–1605.

[6] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[9] "LangGraph documentation," LangChain, Inc. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 6, 2026].

[10] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024.

[11] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[12] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly Transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[13] C. W. J. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.

[14] J. Runge, P. Nowack, M. Kretschmer, S. Flaxman, and D. Sejdinovic, "Detecting and quantifying causal associations in large nonlinear time series datasets," *Science Advances*, vol. 5, no. 11, eaau4996, 2019.

[15] A. Hyvärinen, K. Zhang, S. Shimizu, and P. O. Hoyer, "Estimation of a structural vector autoregression model using non-Gaussianity," *Journal of Machine Learning Research*, vol. 11, pp. 1709–1731, 2010.

[16] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[17] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Quality of Service (IWQoS)*, 2020.

[18] M. A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[19] A. Sharma and E. Kiciman, "DoWhy: An end-to-end library for causal inference," arXiv:2011.04216, 2020.

[20] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[21] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Amsterdam, Netherlands, 2019, pp. 1981–1984.

[22] "Telegram Bot API," Telegram. [Online]. Available: https://core.telegram.org/bots/api. [Accessed: Oct. 6, 2026].